



# Eventix

Event Ticketing Platform

## Part 2 of 2 — **Codebase & File Guide**

What every important file does across the backend and the Android app.

Prepared 14 July 2026 · Author: Bhanu Prakash

## o • The folder map

A monorepo with two halves: a Node/Express API and a native Android app. No web frontend — the API's client is the Android app.

```
Event Ticketing Platform/
├─ backend/                # Node + Express + PostgreSQL API
│  └─ src/
│     │ └─ index.js        # entry: app, routers, boot DB + seed
│     │ └─ db.js           # pg pool, embedded-PG lifecycle, migrations
│     │ └─ auth.js         # register/login + JWT middleware
│     │ └─ events.js        # list/search/detail/create events
│     │ └─ tickets.js       # purchase, checkin, QR sign/verify ★
│     │ └─ seed.js         # demo users + 5 events on first boot
│  └─ migrations/001_init.sql # schema (no_oversell CHECK)
│  └─ loadtest/oversell-test.js # 200-buyer concurrency spec
└─ android/                # Kotlin + Jetpack Compose app
   └─ app/src/main/java/com/etp/app/
      │ └─ EtpApplication.kt # Application + manual DI container
      │ └─ MainActivity.kt   # sole Activity → Compose
      │ └─ ui/Nav.kt         # root nav + role-based bottom bar
      │ └─ data/             # Api, Models, Repository, Session
      │ └─ ui/screens/       # 6 screens (ViewModel in-file)
      │ └─ ui/theme/ · ui/components/
      └─ util/               # Qr.kt (ZXing), Format.kt (₹/dates)
```

**Brand vs package:** the user-facing brand is **Eventix** (strings.xml, splash, auth), while the code namespace/package is `com.etp.app` and the backend is `etp-backend` ("ETP" = Event Ticketing Platform). QR payloads are prefixed `ETP1.` .

# 1 • The backend (backend/src/)

## ENTRY POINT

### backend/src/index.js

Boots the Express app: JSON middleware, mounts the auth/events/tickets routers and `/health`, a central error handler, applies migrations and seeds demo data on startup, then `listen`s on port 3000 with graceful shutdown.

## DATABASE

### backend/src/db.js

Configures the shared `pg.Pool` (max 20), manages the **embedded-Postgres** lifecycle (boots it on :5433 in dev so there's no external DB to install), applies `migrations/001_init.sql` idempotently on boot, and installs a BIGINT→number type parser.

## AUTH

### backend/src/auth.js

The `/register` and `/login` routes (bcrypt + JWT), plus `requireAuth` and `requireOrganizer` middleware and Zod schemas. Issues a 7-day HS256 token carrying the user id + role.

## EVENTS

### backend/src/events.js

List/search (title/venue `ILIKE`), detail, and organizer-only create. A shared `EVENT_SELECT` computes `seatsLeft = capacity - tickets_sold` in SQL so the app never has to.

## ★ CORE — PURCHASES & CHECK-IN

### backend/src/tickets.js

The heart of the project. **Purchase**: a transaction that advisory-locks the event row ( `SELECT ... FOR UPDATE` ), re-checks capacity, inserts the ticket, increments `tickets_sold`, and records the idempotency key — mapping the exclusion/unique violation to HTTP 409. **/me/tickets** returns the wallet with HMAC-signed QR payloads. **/checkin**: an atomic conditional `UPDATE ... WHERE status='valid'` plus timing-safe signature verification.

## SEED

### backend/src/seed.js

On first boot (users table empty) seeds 2 demo users ( `attendee@demo.com`, `organizer@demo.com`, both `password123` ) and 5 Bengaluru-themed events with Unsplash cover images.

#### SCHEMA

**backend/migrations/001\_init.sql**

Tables: `users` (role check attendee/organizer), `events` (capacity, tickets\_sold, `CHECK(tickets_sold <= capacity)`), `tickets` (unique code, status valid/checked\_in), `purchases` (unique idempotency\_key), plus indexes.

#### CONCURRENCY SPEC

**backend/loadtest/oversell-test.js**

Executable proof: 200 parallel buyers / 50 seats, an idempotency replay, a double check-in, and a forged-QR attempt — asserting exactly 50 sales and every guarantee (the run on the previous doc's page).

## 2 • The Android app (com/etp/app/)

### Entry & navigation

| FILE                           | PURPOSE                                                                                                                   |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>EtpApplication.kt</code> | <code>Application</code> + <code>AppContainer</code> (manual DI: session + repository); loads the saved session on start. |
| <code>MainActivity.kt</code>   | The single Activity; edge-to-edge; sets Compose content <code>EtpTheme { RootNav() }</code> .                             |
| <code>ui/Nav.kt</code>         | Root navigation, splash, role-based bottom-nav scaffold, event-detail nav graph.                                          |

### Data layer

| FILE                            | PURPOSE                                                                                                                                              |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>data/Api.kt</code>        | Retrofit <code>EtpApi</code> interface + OkHttp client with a JWT interceptor (attaches <code>Authorization: Bearer</code> from the session).        |
| <code>data/Models.kt</code>     | All <code>@Serializable</code> DTOs (User, Event, Ticket, request/response shapes).                                                                  |
| <code>data/Repository.kt</code> | Wraps API calls in <code>Result&lt;T&gt;</code> , decodes backend error bodies into readable messages, maps <code>IOException</code> → offline hint. |
| <code>data/Session.kt</code>    | <code>SessionManager</code> + <code>SessionState</code> : DataStore-backed JWT/user persistence, exposed as a <code>StateFlow</code> .               |

### Screens (each bundles its own ViewModel)

| FILE                                         | PURPOSE                                                                                                       |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>ui/screens/AuthScreen.kt</code>        | Login/register + role picker ( <code>AuthViewModel</code> ).                                                  |
| <code>ui/screens/HomeScreen.kt</code>        | Discover list, search, pull-to-refresh, seats-left chips ( <code>HomeViewModel</code> ).                      |
| <code>ui/screens/EventDetailScreen.kt</code> | Detail, availability bar, idempotent Buy, success dialog ( <code>EventDetailViewModel</code> ).               |
| <code>ui/screens/TicketsScreen.kt</code>     | Wallet of QR ticket cards, VALID/USED chips ( <code>TicketsViewModel</code> ).                                |
| <code>ui/screens/ScannerScreen.kt</code>     | CameraX + ML Kit QR scanner, permission gate, in-flight guard, result card ( <code>ScannerViewModel</code> ). |
| <code>ui/screens/CreateEventScreen.kt</code> | Organizer event-creation form with date picker + validation ( <code>CreateEventViewModel</code> ).            |

### Support & utils

| FILE                           | PURPOSE                                                                                        |
|--------------------------------|------------------------------------------------------------------------------------------------|
| <code>ui/theme/Theme.kt</code> | Violet Material 3 design system (dark/light schemes, typography, <code>BrandGradient</code> ). |

|                                |                                                                                                                         |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>ui/components/Common.kt</b> | <code>etpViewModel</code> DI helper, CenteredLoader, ErrorState, EmptyState, Chip.                                      |
| <b>util/Qr.kt</b>              | ZXing QR bitmap generator (black-on-white).                                                                             |
| <b>util/Format.kt</b>          | Money (₹) and date/time formatting.                                                                                     |
| <b>app/build.gradle.kts</b>    | SDK levels, the full dependency list, and the <code>BASE_URL</code> BuildConfig ( <code>http://localhost:3000</code> ). |
| <b>AndroidManifest.xml</b>     | INTERNET + CAMERA permissions, cleartext traffic (for localhost), single <code>MainActivity</code> .                    |

## 3 · Running it — cheat sheet

### Backend (near zero-config)

| COMMAND                                        | WHAT IT DOES                                                                                                   |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>cd backend &amp;&amp; npm install</code> | Install deps (embedded Postgres included).                                                                     |
| <code>npm run dev</code>                       | Boots embedded Postgres (:5433) + API on <code>http://localhost:3000</code> , applies schema, seeds demo data. |
| <code>npm run loadtest</code>                  | Runs the 200-buyer concurrency proof (second terminal).                                                        |

Demo logins: attendee `attendee@demo.com` · organizer `organizer@demo.com` · both `password123`.

### Android

| COMMAND                                                               | WHAT IT DOES                                            |
|-----------------------------------------------------------------------|---------------------------------------------------------|
| <code>cd android &amp;&amp; .\gradlew.bat assembleDebug</code>        | Build the debug APK (one is already built in the repo). |
| <code>adb reverse tcp:3000 tcp:3000</code>                            | Tunnel the phone's localhost → the laptop's backend.    |
| <code>adb install -r app\build\outputs\apk\debug\app-debug.apk</code> | Install on a connected device.                          |

**Prereqs:** a physical device is the intended path (the scanner needs a real camera; `adb reverse` makes `localhost:3000` reachable). An emulator would need `10.0.2.2:3000` instead, which requires a one-line source edit.